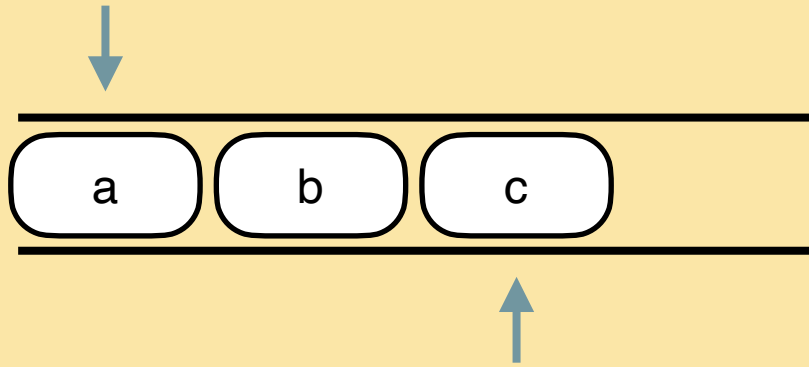




1. 队列的定义和基本操作

队头指针front



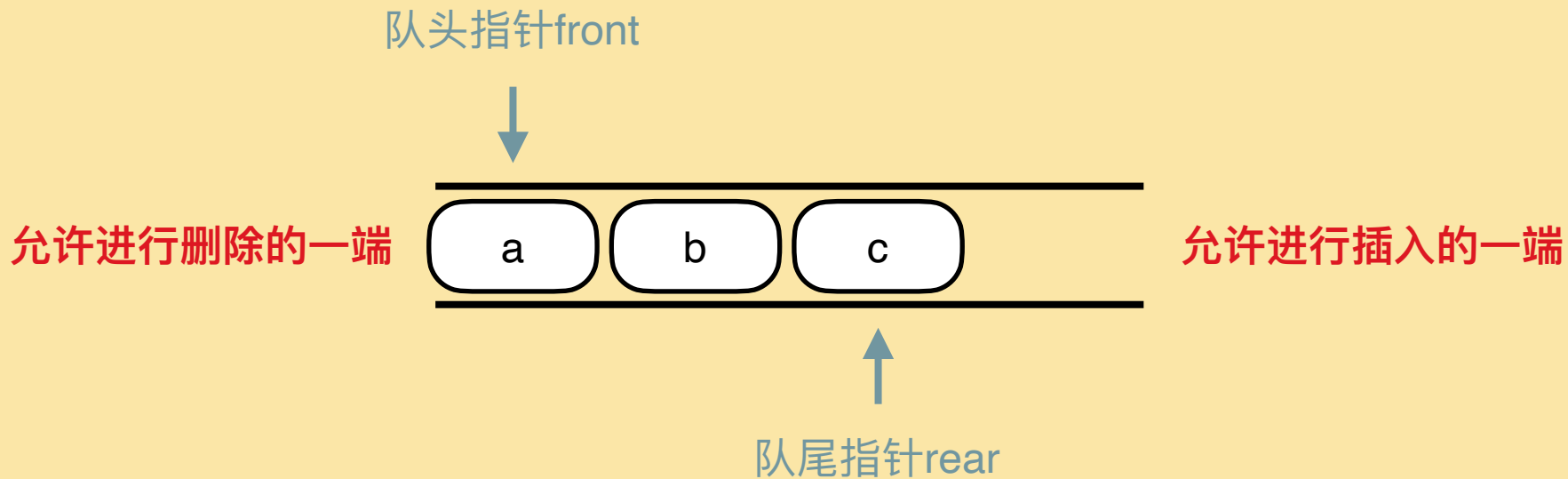
队尾指针rear

队列

只能在一端进行插入，另一端进行删除的线性表



1. 队列的定义和基本操作



队列

只能在一端进行插入，另一端进行删除的线性表



1. 队列的定义和基本操作

队头指针front



队尾指针rear

队列的操作特性：**先进先出** (First In First out, FIFO)

队列 只能在一端进行插入，另一端进行删除的线性表



1. 队列的定义和基本操作

入队A,B,C,D, 再依次出队

队头指针front



队尾指针rear

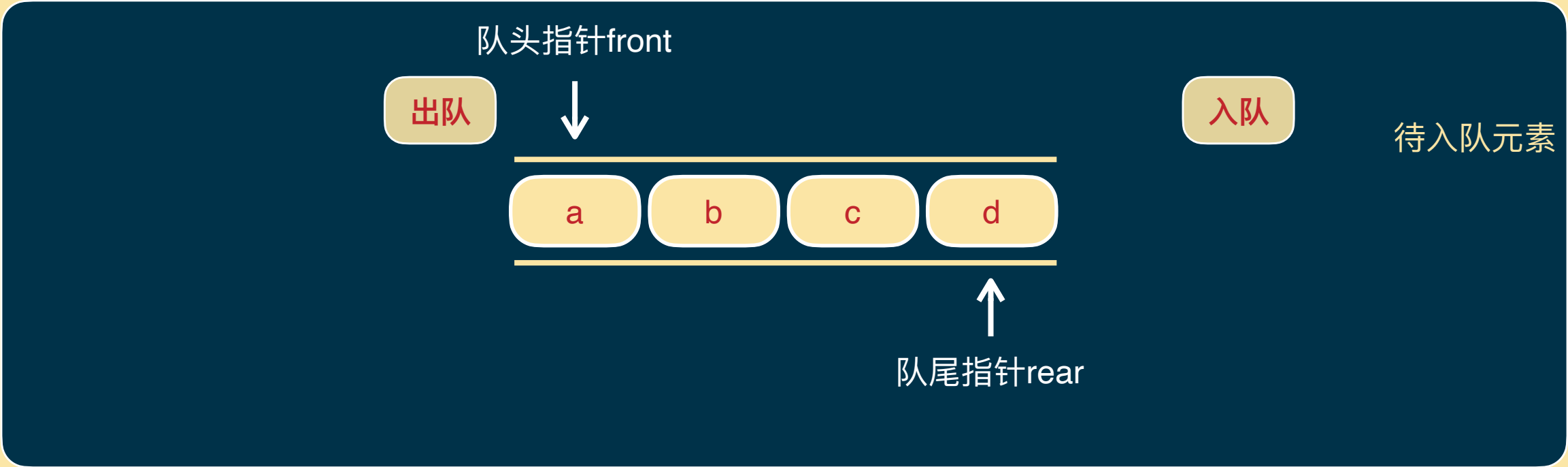
队列的操作特性: **先进先出** (First In First out, FIFO)

队列 只能在一端进行插入, 另一端进行删除的线性表



1. 队列的定义和基本操作

入队A,B,C,D, 再依次出队



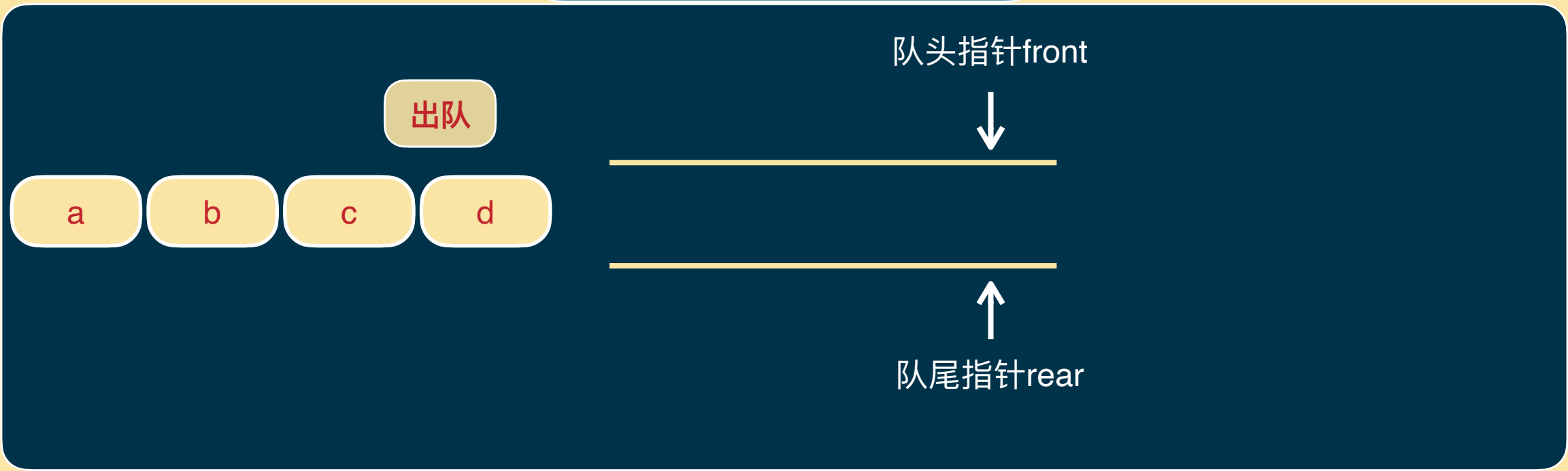
队列的操作特性：**先进先出** (First In First out, FIFO)

队列 只能在一端进行插入，另一端进行删除的线性表



1. 队列的定义和基本操作

入队A,B,C,D, 再依次出队



队列的操作特性：**先进先出** (First In First out, FIFO)

队列 只能在一端进行插入，另一端进行删除的线性表



1. 队列的定义和基本操作

ADT Queue {

数据对象 : $D = \{a_i | a_i \in \text{ElemSet}, i=1, 2, \dots, n, n \geq 0\}$

数据关系 : $R = \{ \langle a_{i-1}, a_i \rangle | a_{i-1}, a_i \in D, i=2, \dots, n \}$

约定其中 a_1 端为队头, a_n 端为队尾。

基本操作 :

InitQueue(& Q)

操作结果 : 构造一个空队列Q。

DestroyQueue(& Q)

初始条件 : 队列Q已存在。

操作结果 : 队列Q被销毁, 不再存在。

ClearQueue(& Q)

初始条件 : 队列Q已存在。

操作结果 : 将Q清为空队列。

QueueEmpty(Q)

初始条件 : 队列Q已存在。

操作结果 : 若Q为空队列, 则返回true, 否则返回false。

QueueLength(Q)

初始条件 : 队列Q已存在。

操作结果 : 返回Q的元素个数, 即队列的长度。

GetHead(Q)

初始条件 : Q为非空队列。

操作结果 : 返回Q的队头元素。

EnQueue(& Q, e)

初始条件 : 队列Q已存在。

操作结果 : 插入元素e为Q的新的队尾元素。

DeQueue(& Q, & e)

初始条件 : Q为非空队列。

操作结果 : 删除Q的队头元素, 并用e返回其值。

QueueTraverse(Q)

初始条件 : Q已存在且非空。

操作结果 : 从队头到队尾, 依次对Q的每个数据元素进行访问。

}ADT Queue



1. 队列的定义和基本操作

队列的基本操作

InitQueue (&Q) : 初始化队列, 构造一个空队列Q。

QueueEmpty (Q) : 判队列空; 若队列Q为空, 返回 true, 否则返回false。

EnQueue (&Q, x) : 入队操作; 若队列Q未满, 将x加入队尾。

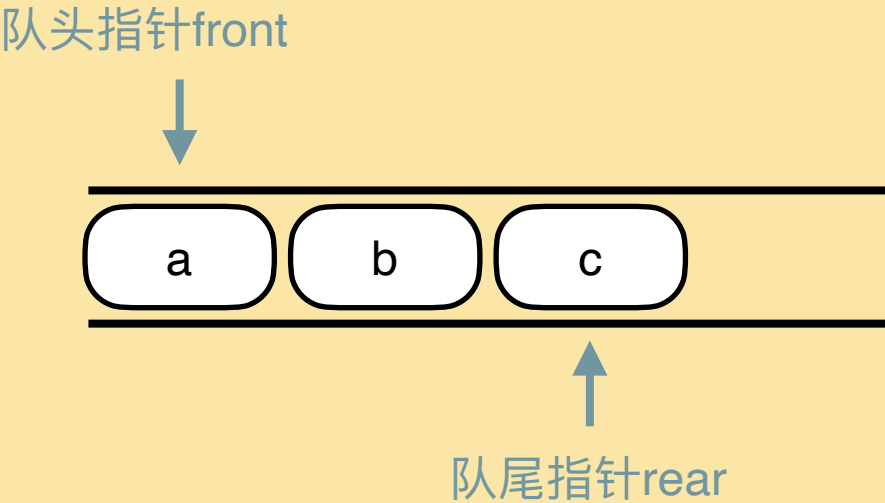
DeQueue (&Q, &x) : 出队操作; 若队列Q非空, 删除队首元素, 并通过x返回其值。

GetHead (Q, &x) : 读队首元素但不出队; 若队列Q非空, 将队首元素赋值给x。

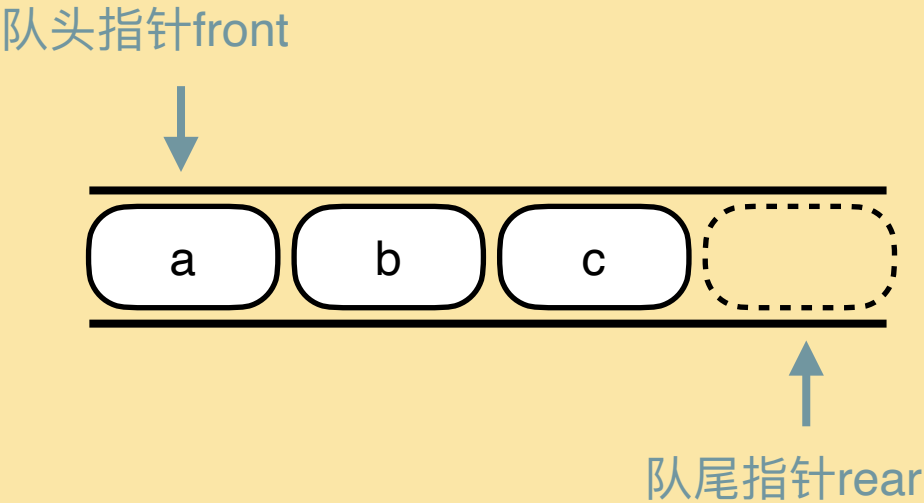


1. 队列的定义和基本操作

指针的不同设置



rear指向队尾元素



rear指向队尾元素的下一个空单元



2. 队列的顺序表示

①顺序队列的定义

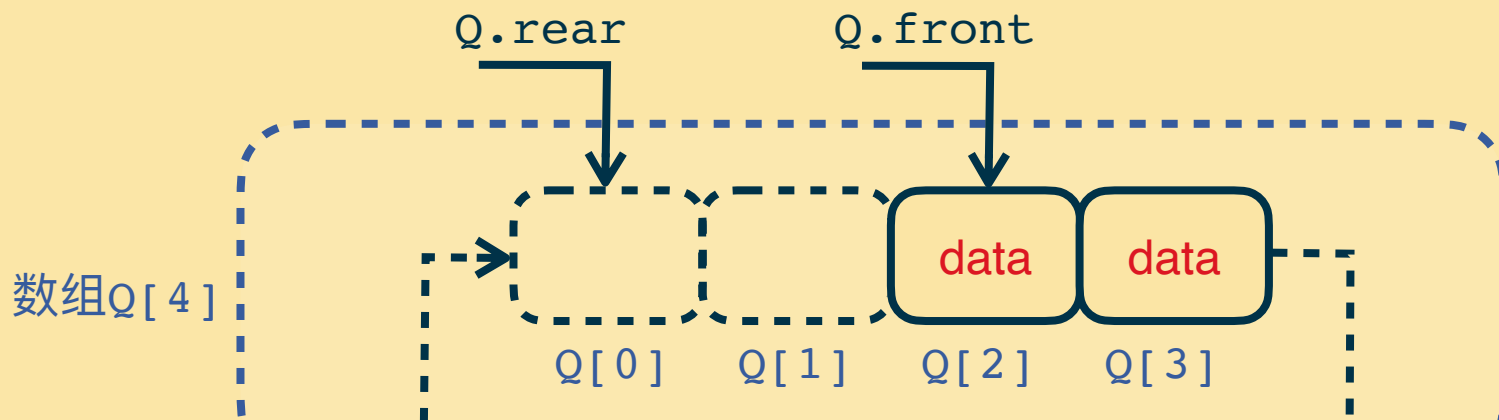
```
#define MaxSize 50 //定义顺序队列的最大容量为50
typedef struct{
    ElemType data[MaxSize]; //用数组data存放队列中的元素
    int front, rear; //front和rear分别是队头指针和队尾指针
}SqQueue; //顺序队列的结构类型为SqQueue
```



2. 队列的顺序表示

②顺序队列的假溢出问题

依次操作：入队，入队，入队，入队，出队，出队



模运算

通过对 n 取模 ($\%n$)，即可使队头、队尾指针在 $0 \sim n-1$ 之间循环变化

为了解决假溢出的问题而引入的首尾相接的队列
一般设置 $front$ 指向队头元素， $rear$ 指向队尾元素的下一个空单元



2. 队列的顺序表示

③循环队列判队空/队满

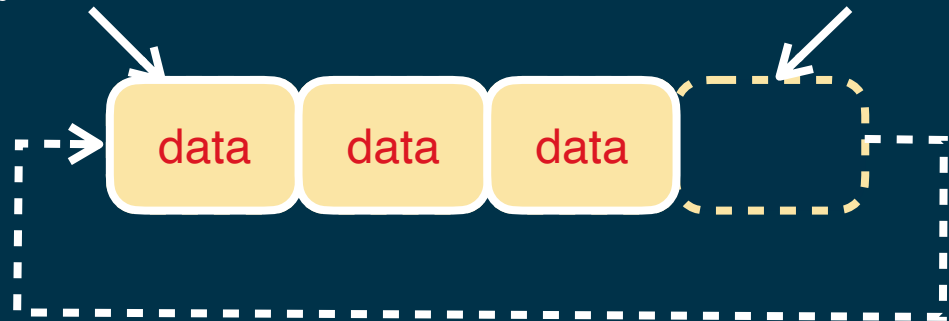
方案1：牺牲一个空单元不放元素

Q.front Q.rear



Q.front

Q.rear



当队列空间大小为m时，仅使用m-1个单元

队空： $Q.front == Q.rear$

队满： $(Q.rear + 1) \% MAXQSIZE == Q.front$

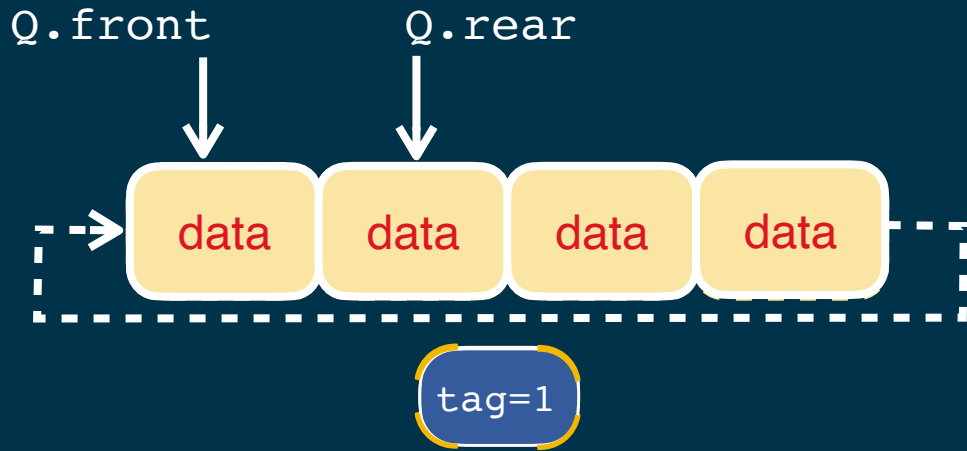
设置front指向队头元素，rear指向队尾元素的下一个空单元



2. 队列的顺序表示

③循环队列判队空/队满

方案2：另设标志位



出队后设置 $tag=0$ ，入队后设置 $tag=1$

队空： $tag == 0, Q.front == Q.rear$

队满： $tag == 1, Q.front == Q.rear$

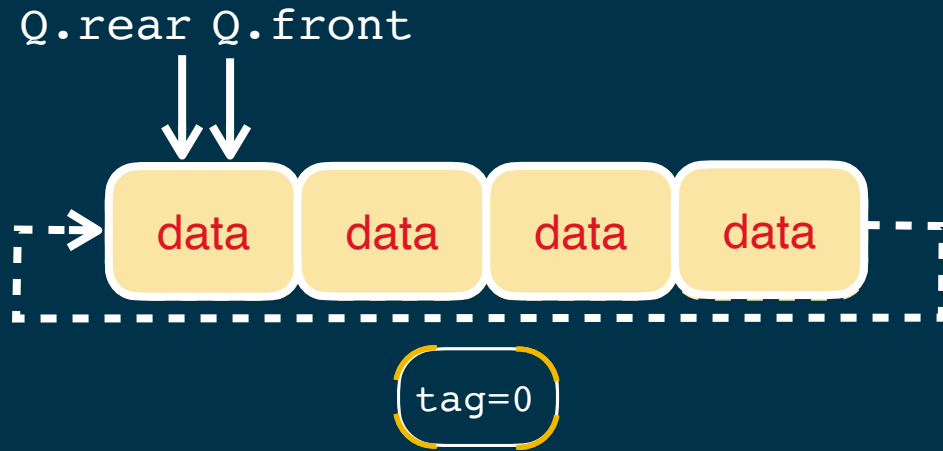
设置 $front$ 指向队头元素， $rear$ 指向队尾元素的下一个空单元



2. 队列的顺序表示

③循环队列判队空/队满

方案2：另设标志位



出队后设置tag=0，入队后设置tag=1

队空：tag == 0, Q.front == Q.rear

队满：tag == 1, Q.front == Q.rear

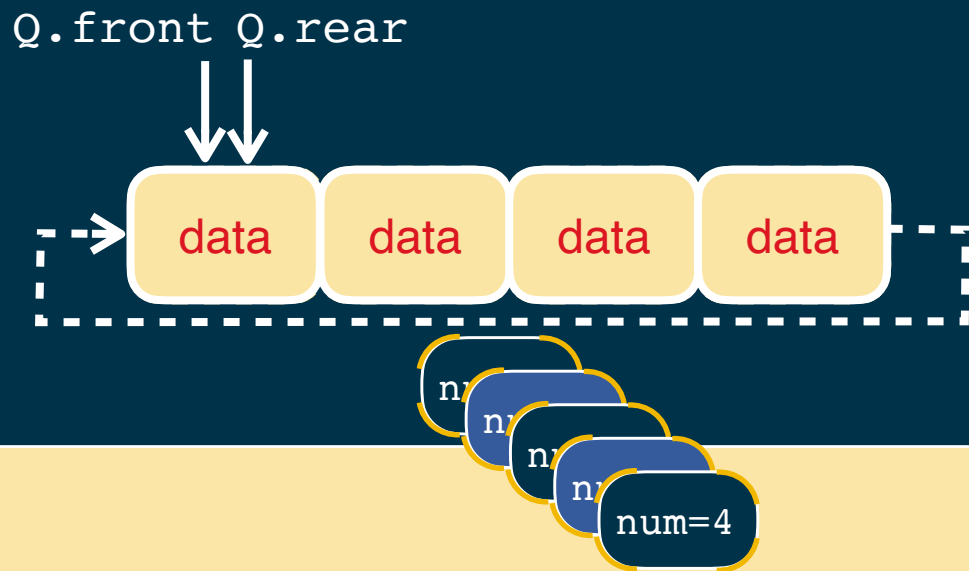
设置front指向队头元素，rear指向队尾元素的下一个空单元



2. 队列的顺序表示

③循环队列判队空/队满

方案3：另设num变量记录队列中元素个数



队空： $\text{num} == 0$, $\text{Q.front} == \text{Q.rear}$

队满： $\text{num} == \text{MAXQSIZE}$, $\text{Q.front} == \text{Q.rear}$

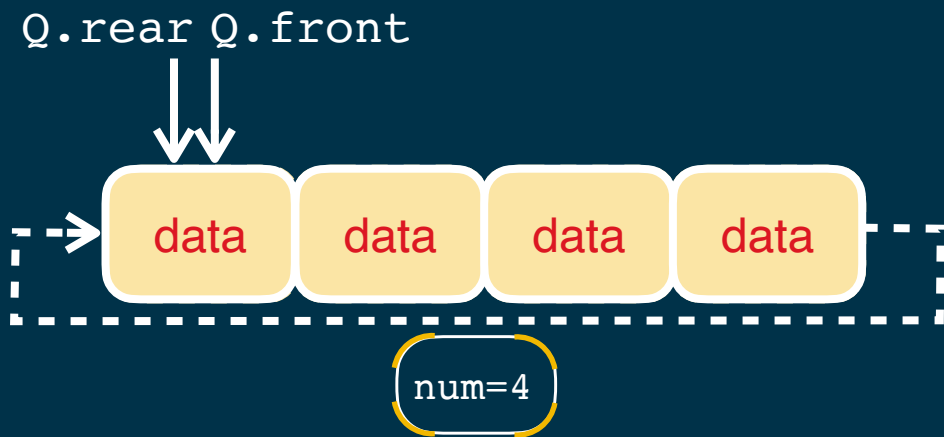
设置front指向队头元素，rear指向队尾元素的下一个空单元



2. 队列的顺序表示

③循环队列判队空/队满

方案3：另设num变量记录队列中元素个数



队空： $\text{num} == 0$, $\text{Q.front} == \text{Q.rear}$

队满： $\text{num} == \text{MAXQSIZE}$, $\text{Q.front} == \text{Q.rear}$

设置front指向队头元素，rear指向队尾元素的下一个空单元



2. 队列的顺序表示

④循环队列的基本操作

初始化

```
void InitQueue(SqQueue &Q){  
    Q.rear = Q.front = 0;  
}
```



2. 队列的顺序表示

④循环队列的基本操作

判队空

```
bool QueueEmpty(SqQueue Q){  
    if(Q.rear == Q.front)  
        return true;  
    else  
        return false;  
}
```



2. 队列的顺序表示

④循环队列的基本操作

入队

```
bool EnQueue(SqQueue &Q, ElemType x){  
    if((Q.rear+1)%MaxSize == Q.front)  
        return false;  
    Q.data[Q.rear] = x;  
    Q.rear = (Q.rear+1)%MaxSize;  
    return true;  
}
```



2. 队列的顺序表示

④循环队列的基本操作

出队

```
bool DeQueue(SqQueue &Q, ElemType &x){  
    if(Q.rear == Q.front)  
        return false;  
    x = Q.data[Q.front];  
    Q.front = (Q.front+1)%MaxSize;  
    return true;  
}
```



3. 队列的链式表示

①链队列的定义

```
typedef struct LinkNode{
    ElemType data;
    struct LinkNode *next;
}LinkNode;
typedef struct{
    LinkNode *front, *rear;
}LinkQueue;
```



3. 队列的链式表示

②链队列的基本操作

初始化

```
void InitQueue(LinkQueue &Q){  
    Q.front = Q.rear = (LinkNode*)malloc(sizeof(LinkNode));  
    Q.front->next = NULL;  
}
```



3. 队列的链式表示

②链队列的基本操作

判队空

```
bool QueueEmpty(LinkQueue Q){  
    if(Q.front == Q.rear)  
        return true;  
    else  
        return false;  
}
```



3. 队列的链式表示

②链队列的基本操作

入队

```
void EnQueue(LinkQueue &Q, ElemType x){  
    LinkNode *s = (LinkNode*)malloc(sizeof(LinkNode));  
    s->data = x;  
    s->next = NULL;  
    Q.rear->next = s;  
    Q.rear = s;  
}
```



3. 队列的链式表示

②链队列的基本操作

出队

```
bool DeQueue(LinkQueue &Q, ElemType &x){
    if(Q.front == Q.rear)
        return false;
    LinkNode *p = Q.front->next;
    x = p->data;
    Q.front->next = p->next;
    if(Q.rear==p)
        Q.rear = Q.front;
    free(p);
    return true;
}
```



4. 双端队列

双端队列

允许在两端进行插入和删除的线性表

已知初始为空的队列Q的一端仅能进行入队操作，另外一端既能进行入队操作又能进行出队操作。
若Q的入队序列是1, 2, 3, 4, 5, 则不能得到的出队序列是

A. 5, 4, 3, 1, 2

B. 5, 3, 1, 2, 4

C. 4, 2, 1, 3, 5

D. 4, 1, 3, 2, 5

祝你今天特别特别开心！